

XDTK: A Cross-Device Toolkit for Input & Interaction in XR

Eric J. Gonzalez*
Google

Khushman Patel
Google

Karan Ahuja
Northwestern University
Google

Mar Gonzalez-Franco†
Google



Figure 1: Vision of a multi-device ecosystem connected within XR.

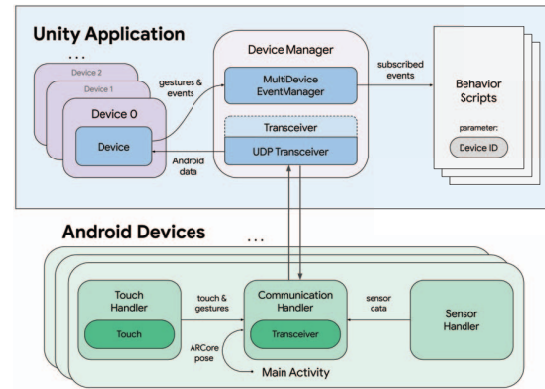


Figure 2: Overview of the XDTK multi-device interaction toolkit.

ABSTRACT

We present XDTK, an open-source Unity/Android toolkit for prototyping multi-device interactions in extended reality (XR). With the Unity package and Android app provided in XDTK, data from any number of devices (phones, tablets, or wearables) can be streamed to and surfaced within a Unity-based XR application. ARCore-supported device also provide self-tracked pose data. Devices on the same local network are automatically discovered by the Unity server and their inputs are routed using a custom event framework. We designed XDTK to be modular and easily extendable to enable fast, simple, and effective prototyping of multi-device experiences by both researchers and developers.

Index Terms: Human-centered computing—User interface toolkits; Human-centered computing—Virtual reality; Human-centered computing—Mixed / augmented reality;

1 INTRODUCTION

Extended reality (XR) platforms arrive at a time in which users are already surrounded by smart devices: from personal (e.g., smartphones and smartwatches), to productivity-oriented (e.g., laptops and tablets), to ambient (e.g., smart speakers). The question of how users will interact with these devices in XR – as well as how they will interact with each other – is not trivial. While there has been considerable research on individual smartphone [17] and tablet [13] use in XR, there has been less focus on XR interactions within a user’s broader multi-device ecosystem. This is in part due to the infrastructure challenges associated with prototyping interactions across various isolated devices (Fig. 1).

Furthermore, there is also the potential to leverage this ecosystem of devices as input peripherals for XR. These devices offer precise

sensing, powerful computation, are familiar to users, and – as in the case of smartphones – are often readily available. Considering that current input methods are often either imprecise (i.e., mid-air hand gestures) or inconvenient (i.e., specialized controllers), there may be a valuable role for existing smart devices to play in the future of XR.

As an initial step toward addressing these challenges, we developed an open-source toolkit¹ for connecting multiple peripheral devices to a central XR application, which we call XDTK: the Cross-Device Toolkit. Here we focus on surfacing inputs and sensor data from the devices to support multi-device input and interactions in XR. Through our Unity package and Android applications, we aim to provide plug-and-play style connectivity as well as a set of extensible primitives which can be used as building blocks for new device-mediated XR experiences. Inspired by the flexibility and customizability of Android, XDTK allows users to easily specify which devices are coupled to which behaviors – for example, an on-screen cursor can be controlled by phone touchscreen normally, but switch to the smartwatch touchscreen when their wrist is raised. In this way, XDTK aims to provide a backbone for exploring transitions between devices, allowing users to create their own device hierarchies.

2 XDTK: THE CROSS-DEVICE TOOLKIT

2.1 Overview

XDTK consists of two key components: a central Unity package which runs on an XR head-mounted display (HMD), and an Android application which can run on any number of peripheral devices. The toolkit handles communication, device discovery, and interaction events through a custom framework. We also provide a set of example behavior scripts, illustrating how to use these inputs. An overview of the system is shown in Fig. 2.

2.2 Communication

Unity connects to peripheral Android devices (phone, tablet, or watch) on the same local network in a server-client architecture using UDP. Once connected to the network, each device begins streaming to the user-provided HMD IP address – this IP address can be set

*e-mail: ejgonz@google.com

†e-mail: margon@google.com

¹Code available at: <https://github.com/google/xdtk>

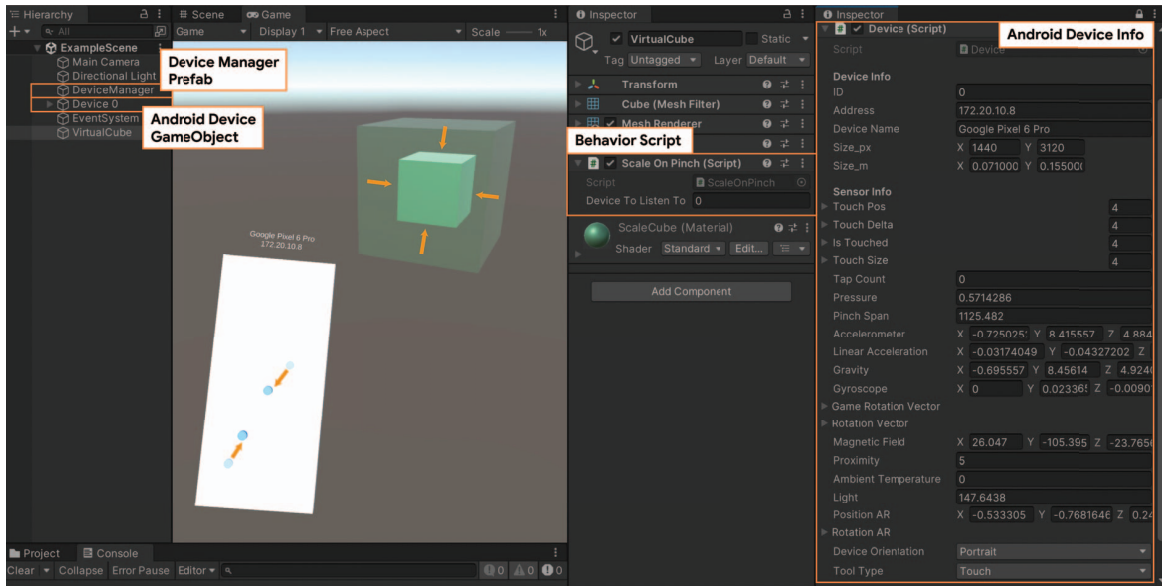


Figure 3: Example XDTK Unity scene highlighting relevant components, including Android device information and behavior scripts.

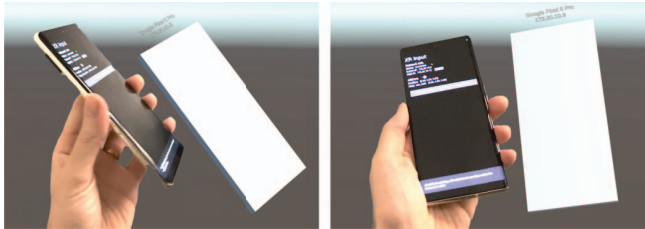


Figure 4: Device IMU controlling the pose of a virtual object.

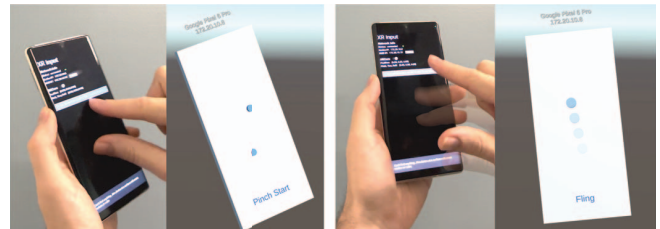


Figure 5: Touch gestures such as pinching (left) and flinging (right).

in-app and is stored by the device between sessions. Following a handshake, the Unity server stores the device's information (e.g. name, address, dimensions) and assigns it a unique ID. Each device then streams sensor data, interaction events, and self-tracked 6DOF pose (if applicable) at a nominal rate of 100 Hz.

2.3 Sensor Data

A core feature of XDTK is the surfacing of standard on-device sensor data from Android devices to drive interactions Unity. The Android application streams data from multiple sensors relevant for interaction, including: accelerometer, IMU, gyroscope, orientation, proximity, and light sensors. The application is easily extensible to include other sensors or forward additional data of any kind.

2.4 Input Events

Most of the existing devices that we interact with have a touch screen (e.g., phones, tablets, and smartwatches). Furthermore, there is an existing vocabulary of interactions that are built on top of standard touch gestures (e.g. tap, double tap, pinch, etc). XDTK enables the use of these familiar gestures within XR (autoreffig:gestures), and allows users remap them across different interactions. Within Unity, these standard gestures are forward as events through a custom event framework. Scripts can subscribe to relevant events in order to create custom behaviors for virtual objects (e.g., pinch to scale an object up or down).

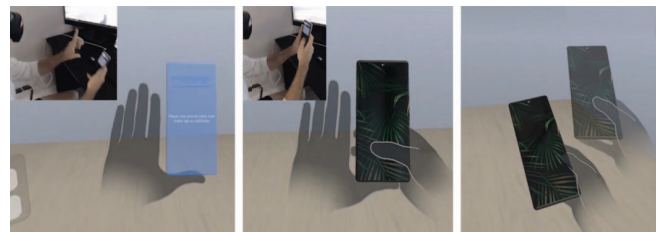


Figure 6: The user frames the phone with an "L shape" (left) and triple taps to calibrate (middle), aligning the device (right).

2.5 Spatial Tracking

For ARCore-supported devices, users have the option of transmitting 6DOF pose and orientation as well, allowing any device to serve as a spatial controller. Importantly, in order to share a common reference frame with the HMD, each device must undergo a calibration step. Given that devices are typically either held or worn, leveraging body tracking from the XR system is a natural way to perform this calibration. While XDTK itself is agnostic to any given XR implementation, we provide an example calibration technique which leverages OpenXR [1] hand tracking: users perform an "L shape" hand gesture, align it with the bottom left corner of the device, and triple tap to calibrate (Fig. 6).

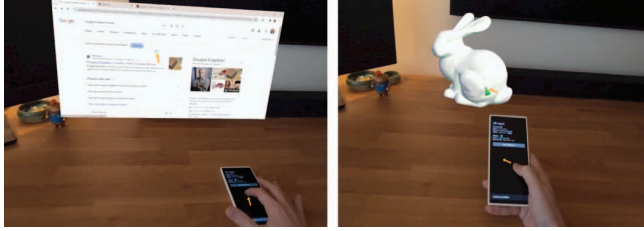


Figure 7: Examples of using a smartphone as a trackpad using XDTK, for a web interface (left) and 3D cursor manipulation (right).

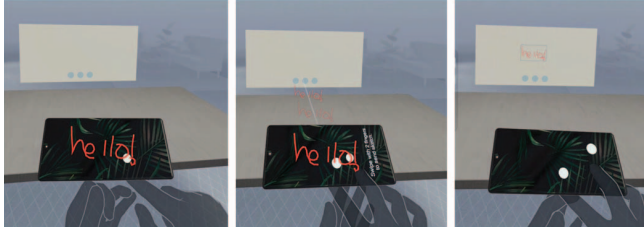


Figure 8: Example of XR whiteboarding built on top of XDTK. A user sketches on their phone (left), flings it to the whiteboard (middle), and adjusts its size using a pinch gesture (right).

2.6 Development Workflow

After importing the XDTK package to Unity, all that is needed to get started is the *DeviceManager* prefab. Upon establishing a new connection, a new *Device* is instantiated. This *Device* object contains all received data from the Android device. In addition, by default a simple to-scale visualization of the device is shown. To allow behaviors to respond to inputs from a specific device, simply specify an ID and call any relevant listeners. By default, each *Device*'s ID is simply the order in which it was paired. Fig. 3 shows a scene in which *Device 0* controls the scale of an object by pinching.

3 EXAMPLE APPLICATIONS

To showcase the capabilities of XDTK, we prepared several prototype applications which we describe here.

3.1 3D Trackpad & Pointer

Modern XR devices are moving away from external controllers (e.g., HTC Vive) to hand tracking input (e.g., Meta Quest). While more convenient and in some cases more natural, this marks a considerable loss in precision and accuracy. For interactions demanding more precise input, such as fine selection and manipulation, smartphones offer a convenient solution. For example, a phone can be used as a trackpad to control a depth-adaptive mouse [16] and select objects in a 3D scene with high precision and minimal effort (Fig. 7).

3.2 Whiteboarding

Smartphones and tablets offer a high-precision touch screens for rich single and multi-touch sensing. Current XR systems, however, are lacking a comparable fine-grained and precise input mechanism. One common scenario where this is evident is collaborative whiteboarding. Here, XDTK can be used to repurpose device touchscreens to provide grounded, precise input [4] to a common whiteboard. Illustrated in Fig. 8, we also show how sketches can be first made privately and then shared to the whiteboard with a fling gesture.

3.3 Sketching with Tablet & Pen

Many tablets come equipped with a stylus or pen for further fine-grained input. Using XDTK we built a simple sketching application



Figure 9: Example of a sketching application which differentiates between pen and touch.

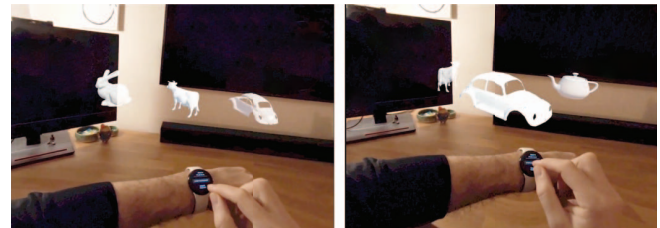


Figure 10: Example of an XR object gallery driven by smartwatch gestures built with XDTK.

(Fig. 9) which can differentiate between touch and pen inputs, allowing for richer sketching interactions, such as manipulating sketches with multi-touch gestures.

3.4 Smartwatch Gestures

Android devices come up with rich vocabulary of interactions built atop of touch gestures such as swipe, tap, double tap, etc. We illustrate the use of a few of these in a 3D model gallery controlled by swipe gestures on a smartwatch (Fig. 10).

4 DISCUSSION

This paper provides a simple but efficient framework to prototype multi-device interaction scenarios in XR. As XR technology advances, and industry trends towards more “natural” inputs such as gaze and hand gestures as primary input mechanisms, the importance of integrating our existing devices as peripheral inputs will grow. Our smartphones and smartwatches are nearly always nearby while using XR – it will be important to fully explore how they might be of use in situations where natural input is insufficient. Perhaps in the future, users will use their phone as their primary XR keyboard [9], or their watch face as an on-demand 3D trackpad, or perhaps they will tap the back of their phone while its in their pocket to discreetly make selections. There is still much to be explored in this space, and towards this we have identified a number of aspects as future work. These would require additional implementations, or perhaps the integration of other open-source projects.

While we focused here on simply bringing device inputs to XR, for fully immersive scenarios it will be important to explore device visualization during interaction more deeply. Hand tracking, for example, degrades when an object such as the phone is being held. Using device touch events to further inform hand pose estimation and/or visualization may be one approach, as explored in *Touching the Droid* [18]. Such device feedback can also be used to further enforce physical constraints during interactions [5, 15]. Regarding the calibration of 6DOF tracking for different devices, it may be possible to leverage information about body kinematics to automate the tracking process, similar to the approach taken by *Pose-on-the-Go* [3]. Another approach may be to enable share mapping data

between devices to improve overall tracking [2], though this would add computational complexity.

Another area this toolkit does not cover is XR-mediated content transfer between devices. Whereas XDTK has initially focused on input streaming, a natural extension is the streaming of content, such as through screen-casting. This would likely require improved wireless communication protocols, such as Bluetooth, Nearby, or WiFi Direct.

It is clear that devices provide more tangible interaction than mid-air gestures, and perhaps offer better ergonomics [4]. However, overall there is still much to be improved in terms of offering rich haptic feedback in XR. There is a rich body of work exploring haptic controllers for XR [6, 7], which can perhaps be leveraged here. Modifying mobile devices with actuated cases is one interesting approach [8, 12], as is the use of miniaturized shape-changing displays [11, 14].

While XDTK increases XR compatibility with a broad range of Android devices including phones, tablets, and watches, many other devices make up the multi-device ecosystem which should ideally integrate with our XR devices. These include headphones [10], assistants, laptops, and IoT devices. However, overall we believe XDTK makes a valuable contribution towards this broader effort and provides a useful framework to begin prototyping multi-device experiences in XR.

5 CONCLUSION

We believe our toolkit XDTK presents a valuable solution advancing rapid prototyping of multi-device ecosystems for XR. By integrating phones, tablets, and wearables with XR experiences, this open-source toolkit not only acknowledges the ubiquity of these devices but also harnesses their advanced sensing and computational capabilities. The seamless wireless communication, sensor integration, and custom Unity framework provided by XDTK represent an important step forward in multi-device XR interaction. We hope researchers and developers use XDTK to build prototypes which explore the power of multi-device interactions and ultimately help shape the role our devices will play inside XR.

ACKNOWLEDGMENTS

The authors wish to acknowledge the use of generative AI tools to aid in the development of Figure 1.

REFERENCES

- [1] Openxr - a royalty-free standard for virtual reality and augmented reality. <https://www.khronos.org/openxr/>.
- [2] K. Ahuja, M. Goel, and C. Harrison. Bodyslam: Opportunistic user digitization in multi-user ar/vr experiences. In *Proceedings of the 2020 ACM Symposium on Spatial User Interaction*, pp. 1–8, 2020.
- [3] K. Ahuja, S. Mayer, M. Goel, and C. Harrison. Pose-on-the-go: Approximating user pose with smartphone sensor fusion and inverse kinematics. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–12, 2021.
- [4] Y. F. Cheng, T. Luong, A. R. Fender, P. Streli, and C. Holz. Comfortable user interfaces: Surfaces reduce input error, time, and exertion for tabletop and mid-air user interfaces. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 150–159. IEEE, 2022.
- [5] B. A. Cohn, A. Maselli, E. Ofek, and M. Gonzalez-Franco. Snapmove: Movement projection mapping in virtual reality. In *2020 IEEE International Conference on Artificial Intelligence and Virtual Reality (AIVR)*, pp. 74–81. IEEE, 2020.
- [6] E. J. Gonzalez, E. Ofek, M. Gonzalez-Franco, and M. Sinclair. X-rings: A hand-mounted 360° shape display for grasping in virtual reality. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, UIST '21, p. 732–742. Association for Computing Machinery, New York, NY, USA, 2021. doi: 10.1145/3472749.3474782
- [7] M. Gonzalez-Franco, E. Ofek, C. Holz, A. Steed, J. Lanier, B. Buxton, K. Hinckley, and M. Sinclair. Taxonomy of hand-object haptics for virtual reality. *TechRxiv*, 2023.
- [8] S. Jang, L. H. Kim, K. Tanner, H. Ishii, and S. Follmer. Haptic edge display for mobile tactile interaction. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pp. 3706–3716, 2016.
- [9] F. Matulic, A. Ganeshan, H. Fujiwara, and D. Vogel. Phonetroller: Visual representations of fingers for precise touch input with mobile phones in vr. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, 2021.
- [10] P. Panda, M. J. Nicholas, D. Nguyen, E. Ofek, M. Pahud, S. Rintel, M. Gonzalez-Franco, K. Hinckley, and J. Lanier. Beyond audio: Towards a design space of headphones as a site for interaction and sensing. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, pp. 904–916, 2023.
- [11] A. F. Siu, E. J. Gonzalez, S. Yuan, J. B. Ginsberg, and S. Follmer. Shapeshift: 2d spatial manipulation and self-actuation of tabletop shape displays for tangible and haptic interaction. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, p. 1–13. Association for Computing Machinery, New York, NY, USA, 2018. doi: 10.1145/3173574.3173865
- [12] E. Strasnick, J. Yang, K. Tanner, A. Olwal, and S. Follmer. shiftio: Reconfigurable tactile elements for dynamic affordances and mobile interaction. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, pp. 5075–5086, 2017.
- [13] H. B. Surale, A. Gupta, M. Hancock, and D. Vogel. Tabletinvr: Exploring the design space for using a multi-touch tablet in virtual reality. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, 2019.
- [14] R. Suzuki, E. Ofek, M. Sinclair, D. Leithinger, and M. Gonzalez-Franco. Hapticbots: Distributed encountered-type haptics for vr with multiple shape-changing mobile robots. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, pp. 1269–1281, 2021.
- [15] Y. Tao, C. Y. Wang, A. D. Wilson, E. Ofek, and M. Gonzalez-Franco. Embodying physics-aware avatars in virtual reality. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–15, 2023.
- [16] Q. Zhou, G. Fitzmaurice, and F. Anderson. In-depth mouse: Integrating desktop mouse into virtual reality. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pp. 1–17, 2022.
- [17] F. Zhu and T. Grossman. BISHARE: Exploring Bidirectional Interactions Between Smartphones and Head-Mounted Augmented Reality. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI '20, pp. 1–14. Association for Computing Machinery, New York, NY, USA, 2020. event-place: Honolulu, HI, USA. doi: 10.1145/3313831.3376233
- [18] F. Zhu, Z. Lyu, M. Sousa, and T. Grossman. Touching the droid: Understanding and improving touch precision with mobile devices in virtual reality. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 807–816. IEEE, 2022. doi: 10.1109/ISMAR55827.2022.00099